
Addis Ababa University

Department of Physics

# Computational Physics

## Activity Solutions

*Submitted by:*
**Getabalew Manegerew**
ID: UGR/6981/15

*Instructor:*
**Dr. Kenate**

January 1, 2026

# Problem 1: Compare-ODE Driver

## Problem Statement

Construct a `compare-ode` driver using Euler, Picard, and Predictor-Corrector subroutines to solve:

a) $\frac{d^2y}{dx^2} + \beta\frac{dy}{dx} + \omega_0^2 y = 0$

b) $\frac{d^2y}{dx^2} + \beta\frac{dy}{dx} + \omega_0^2 y = \eta(t)$

**Parameters:** $\beta = 0.5$, $y(0) = 0.5$, $\dot{y}(0) = 1.0$, $\omega_0 = 1.0$, and $\eta \in [-1, 1]$.

## Fortran Code

```
MODULE ODE_DRIVER
    CONTAINS
    SUBROUTINE euler(x, y, z, dx)
    IMPLICIT NONE
    REAL, EXTERNAL :: f
    REAL, DIMENSION(0:) :: x, y, z
    REAL :: dx
    INTEGER :: i, k


    k = size(x) - 1
    DO i=1, k
        x(i) = x(i-1) + dx
        y(i) = y(i-1) + dx*z(i-1)
        z(i) = z(i-1) - dx*f(y(i-1), z(i-1))
    END DO

END SUBROUTINE euler

SUBROUTINE picard(x, y, z, dx)
    IMPLICIT NONE
    REAL, EXTERNAL :: f
    REAL, DIMENSION(0:) :: x, y, z
    REAL :: dx, k1, k2, p1, p2
    INTEGER :: i, N
    N = size(x) - 1

    DO i=1, N
        x(i) = x(i-1) + dx
        y(i) = y(i-1) + dx*z(i-1)
        z(i) = z(i-1) + dx*f(y(i-1), z(i-1))

        k1 = dx*z(i-1)
        k2 = dx*z(i)
        p1 = dx*f(y(i-1), z(i-1))

```

```
37          y(i) = y(i-1) + (k1 + k2)/2
38          p2 = dx*f(y(i), z(i))
39          z(i) = z(i-1) + (p1 + p2)/2
40      END DO
41
42  END SUBROUTINE picard
43
44  SUBROUTINE corrector(x, y, z, dx)
45      IMPLICIT NONE
46      REAL, EXTERNAL :: f
47      REAL, DIMENSION(0:) :: x, y, z
48      REAL :: dx, k1, k2, k3, p1, p2, p3
49      INTEGER :: i, N
50      N = size(x) - 1
51
52       DO i=1, N - 1
53          x(i) = x(i-1) + dx
54          y(i) = y(i-1) + dx*z(i-1)
55          z(i) = z(i-1) + dx*f(y(i-1), z(i-1))
56
57          k1 = dx*z(i-1)
58          k2 = dx*z(i)
59          p1 = dx*f(y(i-1), z(i-1))
60
61          y(i+1) = y(i-1) + 2*k2
62          p2 = dx*f(y(i), z(i))
63          z(i+1) = z(i-1) + 2*p2
64
65          k3 = dx*z(i+1)
66          p3 = dx*f(y(i+1), z(i+1))
67          y(i+1) = y(i-1) + (k1 + 4*k2 + k3)/3
68          z(i+1) = z(i-1) + (p1 + 4*p2 + p3)/3
69
70      END DO
71
72  END SUBROUTINE corrector
73
74
75  END MODULE ODE_DRIVER
76
77  PROGRAM problem3_1a
78      USE ODE_DRIVER
79      IMPLICIT NONE
80      INTEGER, PARAMETER :: N=1000
81      REAL :: x(0:N), y_e(0:N), z_e(0:N),x_p(0:N), y_p(0:N), z_p(0:
            N), x_c(0:N), y_c(0:N), z_c(0:N), dx, a=0, c=1
82      INTEGER :: k, i
83
84      ! let dy/dx = z, and a & c the range of the solution
85      ! we also define external function " f " to be used as dz/dx
            = f
```

```
86
87      dx = (c-a)/(N)
88      x(0) = 0
89      y_e(0) = 0.5
90      z_e(0) = 1
91      y_p(0) = 0.5
92      z_p(0) = 1
93      y_c(0) = 0.5
94      z_c(0) = 1
95
96      CALL euler(x, y_e, z_e, dx)
97      CALL picard(x, y_p, z_p, dx)
98      CALL corrector(x, y_c, z_c, dx)
99
100     OPEN(UNIT=10, FILE="problem31a.csv", POSITION="APPEND",
           ACCESS="SEQUENTIAL", STATUS="REPLACE")
101     DO k=0, N-2
102         WRITE(UNIT=10, FMT="(7f9.5)") x(k),y_e(k), z_e(k), y_p(k)
              , z_p(k), y_c(k), z_c(k)
103     END DO
104     CLOSE(UNIT=10)
105
106
107 END PROGRAM problem3_1a
108
109 FUNCTION f(y,z) RESULT(s)
110     IMPLICIT NONE
111     REAL :: z, y, s, b=0.5, w0=1, r
112     CALL RANDOM_NUMBER(r)
113
114     s = -(b*z + w0**2*y) + 2*r - 1
115 END FUNCTION f
```

Listing 1: Fortran code for ODE solvers

## Results and Graphs

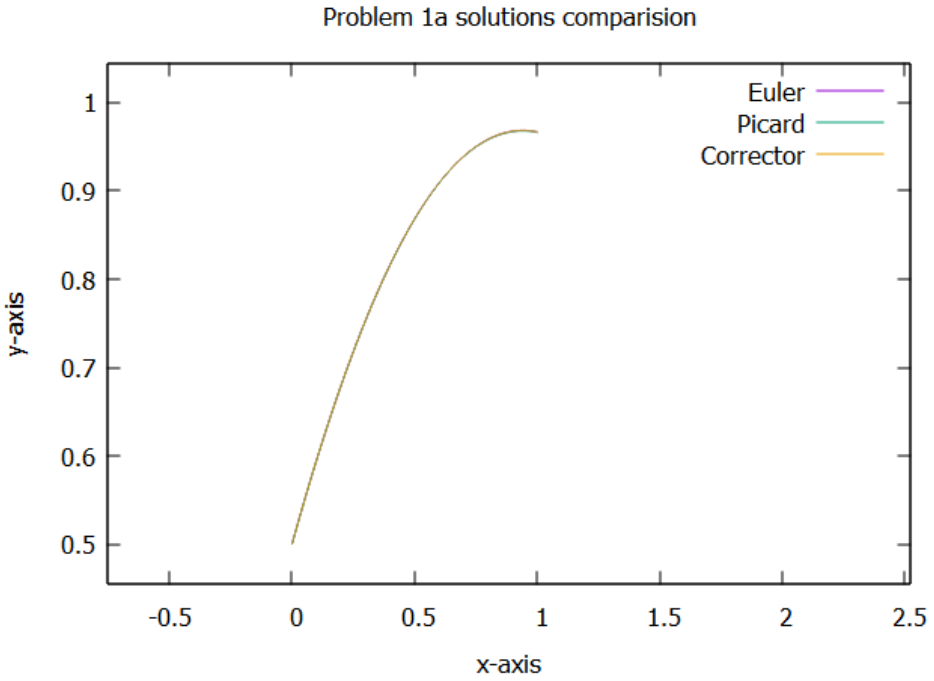


Figure 1: Solution for the homogeneous ODE (Part a) using different methods. The plot is x versus y as both given in the equation.

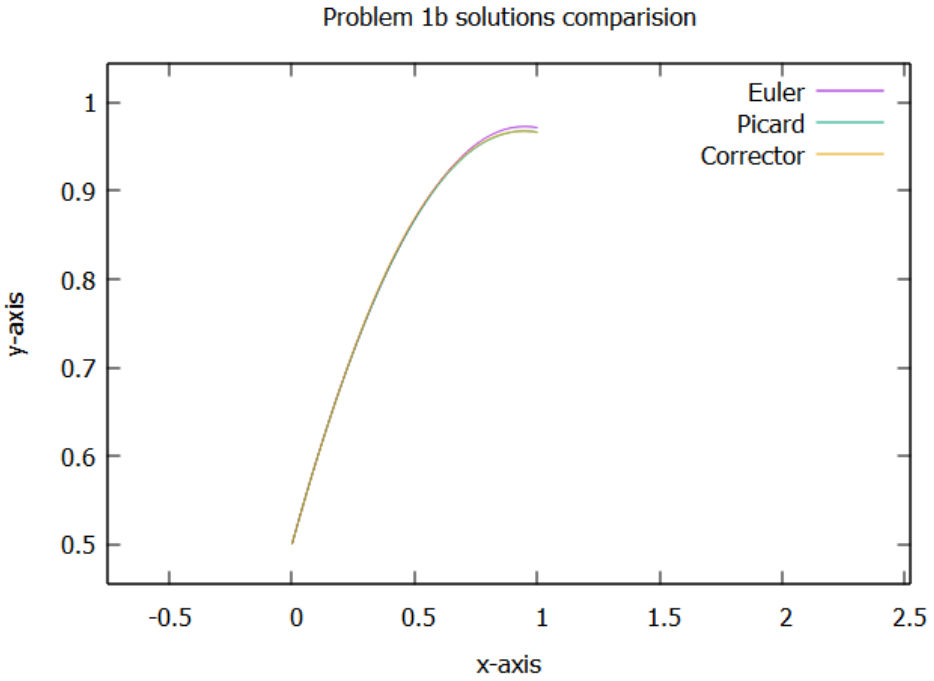


Figure 2: Solution for the forced ODE with random noise (Part b). The plot is x versus y as both given in the equation.

# Problem 2: Compare-Roots Driver

## Problem Statement

Find the zeros of the function using Newton and Bisection subroutines:

$$f(x) = -x^2 - 2\ln(x) - 3e^{-x}$$

**Range:** $[a, b] = [0.2, 2.2]$ with tolerance $\varepsilon = 2.0 \times 10^{-5}$.

## Fortran Code

```
MODULE ROOT_DRIVER
    IMPLICIT NONE
    CONTAINS
    SUBROUTINE bisection(a, b, x, e, m)
        REAL, EXTERNAL :: f
        REAL :: a, b, x, e
        INTEGER :: m
        x = (a+b)/2

    DO WHILE(abs(f(x))>e)
        x = (a+b)/2
        IF ((f(a)*f(x))<0) THEN
            b=x
        ELSE
            a=x
        END IF
        m=m+1
    END DO

    END SUBROUTINE bisection

    SUBROUTINE newton(a,x, e, dx, n)
        REAL, EXTERNAL :: f, fd
        REAL :: a, x, e, dx
        INTEGER :: n
        n=0
        x = a

    DO WHILE(f(x)>e)
        x = a + f(a)/fd(x)
        a = a + dx
        n=n+1
    END DO

    END SUBROUTINE newton


END MODULE ROOT_DRIVER

```

```
40
41 PROGRAM problem3_2
42     USE ROOT_DRIVER
43     IMPLICIT NONE
44     REAL, EXTERNAL :: f, fd
45     INTEGER, PARAMETER :: N=100
46     REAL :: a=0.2, b=2.2, x, e=2e-5, dx=1e-5
47     INTEGER :: m, k
48
49
50
51     CALL bisection(a,b,x,e,m)
52     PRINT*, "Zero of f in Bisection methods", x, "Number of
          iteration in Bisection method" , m
53     CALL newton(a,x,e,dx, k)
54     PRINT*, "Zero of f in Newton's methods", x, "Number of
          iteration in Newton's method", k
55
56
57
58 END PROGRAM problem3_2
59
60 FUNCTION f(x) RESULT(y)
61     IMPLICIT NONE
62     REAL :: x, y
63     y = -x**2 - 2*log(x) - 3*exp(-x)
64 END FUNCTION f
65
66 FUNCTION fd(x) RESULT(Y)
67     IMPLICIT NONE
68     REAL :: x, y
69     y = -2*x - 2/x + 3*exp(-x)
70 END FUNCTION
```

Listing 2: Fortran code for Root Finding

## Numerical Results

The roots found by the program are:

- **Newton Method:** $x \approx 0.319595486$ (Iterations: 3)
- **Bisection Method:** $x \approx 0.319583148$ (Iterations: 17)

# Problem 3: Compare-Differentiate Driver

## Problem Statement

Differentiate the following function using Forward (FD), Backward (BD), and Central/Midpoint (MD) difference subroutines:

$$f(x) = x^3 + 2x^2 + 2x + 2$$

**Range:** $[-5, 5]$.

## Fortran Code

```
MODULE DIFFERENTIATION_DRIVER
    IMPLICIT NONE

    CONTAINS
    SUBROUTINE backward(x,bd,dx)
        IMPLICIT NONE
        REAL, EXTERNAL :: f
        REAL, DIMENSION(0:) :: x, bd
        REAL :: dx
        INTEGER :: i, N
        N = size(x)

        DO i=1, N
            x(i) = x(i-1) + dx
            bd(i) = (f(x(i)) - f(x(i-1)))/dx
        END DO

    END SUBROUTINE backward

    SUBROUTINE forward(x, fd, dx)
        IMPLICIT NONE
        REAL, EXTERNAL :: f
        REAL :: x(0:), fd(:), dx
        INTEGER :: i, N
        N = size(fd)

        DO i=1, N
        x(i) = x(i-1) + dx
        END DO

        DO i=1, N
            fd(i) = (f(x(i+1)) - f(x(i)))/dx
        END DO

    END SUBROUTINE forward

    SUBROUTINE midpoint(x,md,dx)
        IMPLICIT NONE
```

```
39          REAL, EXTERNAL :: f
40          REAL :: x(0:), md(:), dx
41          INTEGER :: i, N
42          N = size(md)
43
44          DO i=1, N
45              x(i) = x(i-1) + dx
46          END DO
47
48          DO i=1, N-1
49              md(i) = (f(x(i+1)) - f(x(i-1)))/(2*dx)
50          END DO
51
52      END SUBROUTINE midpoint
53
54
55  END MODULE DIFFERENTIATION_DRIVER
56
57  PROGRAM problem33
58      USE DIFFERENTIATION_DRIVER
59      IMPLICIT NONE
60      REAL, EXTERNAL :: f
61      INTEGER, PARAMETER :: N=10000
62      REAL :: x(0:N), fd(1:N), bd(1:N-1), md(1:N-1), a=-5, b=5, dx
63      INTEGER :: i, k
64      dx = (b-a)/N
65      x(0) = a
66
67
68
69
70      CALL backward(x, bd, dx)
71      CALL forward(x, fd, dx)
72      ! Note: Ensure you meant to call midpoint here, not forward
            twice
73      CALL midpoint(x, md, dx)
74
75
76      OPEN(UNIT=10, FILE="problem33.csv", POSITION="APPEND", ACCESS
           ="SEQUENTIAL", STATUS="REPLACE")
77      DO k=1, N-1
78      WRITE(UNIT=10, FMT="(4f10.5)") x(k), bd(k), fd(k), md(k)
79      END DO
80      CLOSE(UNIT=10)
81
82
83
84  END PROGRAM problem33
85
86  FUNCTION f(x) RESULT(y)
87      IMPLICIT NONE
```

```
88      REAL :: x, y
89      y = x**3 + 2*x**2 + 2*x + 2
90  END FUNCTION f
```

Listing 3: Fortran code for Numerical Differentiation

## Results and Graphs

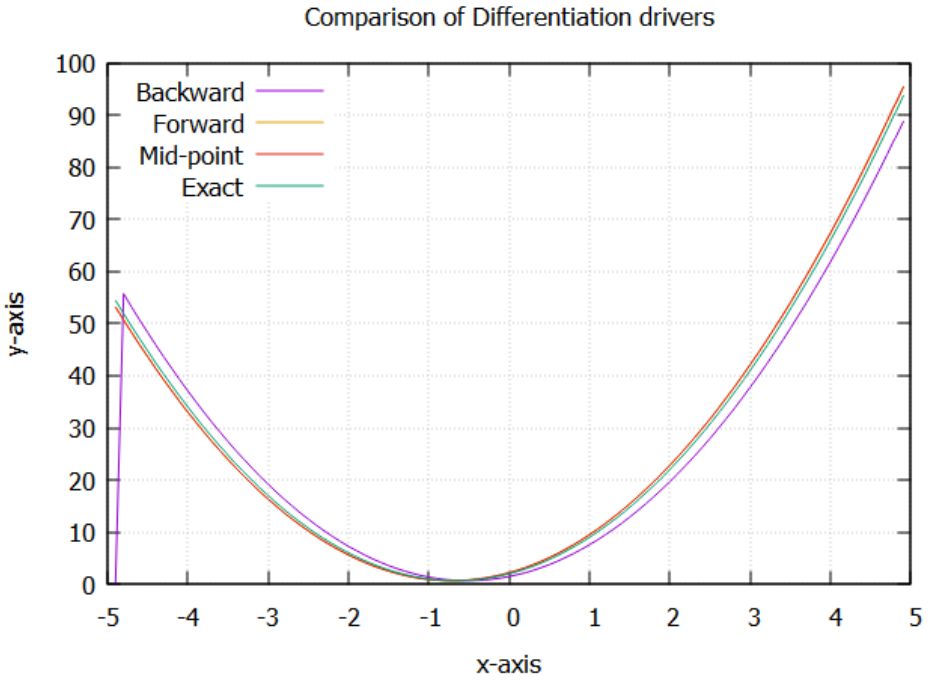


Figure 3: Comparison of Analytical Derivative vs FD, BD, and MD numerical methods.